Omega TK Chatbot

A Complete Technical Breakdown

This document explains how the Omega TK Chatbot works — from the moment a user types a question to the moment a validated Python code snippet appears on screen. It covers the backend pipeline, the three-layer safety system, the AI routing logic, the React frontend, and where the project can grow from here.

FastAPI | OpenAI GPT-4o-mini | FAISS | React + Vite | Supabase

What This Project Does

The Omega TK Chatbot is a domain-specific AI assistant built for the OpenEye Omega Toolkit — a Python library used in computational chemistry and drug discovery for generating 3D molecular conformers. Instead of a generic chatbot, this system knows exactly what it is supposed to do: answer questions, explain concepts, and write correct, validated Python code using the OpenEye API.

It is not simply a wrapper around ChatGPT. Every response passes through multiple safety layers that check intent, verify retrieval confidence, and validate generated code before it ever reaches the user. If the AI hallucinates a fake function name, the system catches it and retries automatically.

Contents

01

System Architecture — The Big Picture

Before diving into any code, it helps to see the whole system at once. The chatbot has three major parts that work together: a React frontend that the user talks to, a FastAPI backend that processes every message, and a set of external services (OpenAI for AI, FAISS for search, and Supabase for storage).

Layer	Technology	What It Does
Frontend	React 18 + Vite + Tailwind CDN	The chat UI, welcome screen, analytics dashboard, and knowledge panel the user interacts with.
API Server	FastAPI + Python 3.12	Receives every message, runs it through the full pipeline, and returns a structured JSON response.
RAG Engine	FAISS + text-embedding-3-small	Converts the user's question into a vector and finds the most relevant chunks from the pre-built documentation index (131 vectors).
LLM	GPT-4o-mini (OpenAI)	Classifies intent, generates conversational responses, writes code, and summarizes long conversations.
Validator	ast + regex (Python stdlib)	Catches syntax errors, missing OpenEye patterns, and hallucinated API names in generated code — no LLM needed.
Database	Supabase (PostgreSQL)	Stores chat history, query analytics, thumbs up/down feedback, and user-uploaded knowledge chunks.

The key design decision: the documentation was pre-processed offline. All the OpenEye Omega Toolkit docs were chunked into 131 text fragments and converted into embedding vectors using OpenAI's text-embedding-3-small model. These are stored in a FAISS index on disk. At query time, only the most relevant fragments are retrieved — the LLM never has to read the full docs.

02

How a Request Travels Through the Backend

Every message the user sends follows the exact same journey. Here is that journey, step by step, from the moment the user hits Send to when they see a response.

1	**User Sends Message**	The React frontend posts the message to POST /api/chat along with the conversation history (last 6 turns) and a session_id stored in the browser's localStorage.
2	**Pre-Guardrail Check**	Before any LLM call, the server checks if the message is a greeting, a thank-you, or a capabilities question. If yes, a hardcoded response is returned instantly — zero latency, zero API cost.
3	**Layer 1 — Intent Filter**	A keyword blocklist and allowlist checks whether the question is on-topic. Queries containing words like 'poem', 'weather', 'ignore your instructions' are rejected immediately with a polite fallback message.
4	**LLM Intent Classifier**	GPT-4o-mini classifies the surviving query into one of four buckets: GREETING, CONVERSATION, CODE, or OFF_TOPIC. This call uses only 10 tokens and runs in roughly 300ms. It routes the request to the correct generation path.
5	**FAISS Retrieval (Layer 2)**	The query is embedded and compared against the 131-vector FAISS index. The top matching documentation chunks are retrieved. If no chunk scores above the threshold (0.20 for concepts, 0.30 for code), the request is rejected with a 'not enough information' fallback.
6	**Knowledge Base Merge**	If the user has uploaded any personal notes or files via the Knowledge Base panel, relevant chunks from their uploads are retrieved using cosine similarity in Python and merged into the context alongside the FAISS results.
7	**Generation — Path A or B**	CONVERSATION intent: GPT-4o-mini generates a natural-language explanation using a system-message prompt. CODE intent: GPT-4o-mini generates code using a stricter prompt that demands the 5-step OpenEye pattern.

8	**Layer 3 — Code Validation**	For code responses only, the generated code is validated: syntax check (ast.parse), pattern check (regex confirms the 5-step workflow), and API name check (allowlist catches hallucinated function names). If validation fails, the error is fed back to GPT as a retry hint — up to 5 attempts.
9	**Supabase Logging**	The final response, intent, FAISS score, and latency are written to queries_log in Supabase. The chat turn is saved to chat_history. Both writes are fire-and-forget — they don't block the response.
10	**Response to Client**	A JSON object is returned: { explanation, code, language, is_fallback, fallback_message, attempts }. The frontend renders the explanation as text and the code in a syntax-highlighted block with a Copy button.

03

Layer 1 — Intent Guard

The first line of defence is a pure Python keyword filter. It runs in microseconds and costs nothing because it makes no API calls. The logic is straightforward: if the query contains any word from the blocklist, reject it. If it contains any word from the allowlist, pass it through. If neither list matches, the request is allowed through anyway — to avoid false rejections for legitimate but unusually phrased questions.

Blocklist (INVALID signals)	Allowlist (VALID signals)
poem, poetry, joke, story, song, weather, news, recipe, food, movie, ignore, forget, pretend, roleplay, bypass	how to, generate, create, explain, what is, code, script, configure, write, show me, example, implement, use

Real example: A user types 'write a poem about molecules.' The word 'poem' is in the blocklist, so it is rejected before any LLM call is made. The user gets a polite message explaining the assistant only handles Omega TK topics. Total processing time: under 1ms.

04

Layer 2 — FAISS Retrieval

RAG stands for Retrieval-Augmented Generation. Instead of asking GPT to answer from memory (which leads to hallucinations), the system first finds relevant passages from the actual OpenEye documentation and includes them in the prompt. GPT then answers based on what it can read, not what it vaguely recalls.

How the Index Was Built (Offline, Done Once)

1	**Collect Docs**	The OpenEye Omega Toolkit documentation was gathered: API references, tutorials, code examples.
2	**Chunk Text**	Each document was split into overlapping chunks of roughly 400 characters with 50-character overlap so context is not lost at boundaries.
3	**Embed**	Each chunk was sent to OpenAI's text-embedding-3-small model, which returns a 1536-dimensional vector representing the meaning of that chunk.
4	**Index**	All vectors were loaded into a FAISS IndexFlatIP (inner product) index. The index was saved to disk as a .faiss file alongside a chunks.json metadata file.

How Retrieval Works at Query Time

1	**Embed Query**	The user's question is embedded using the same text-embedding-3-small model, producing a 1536-dim vector.
2	**Search Index**	FAISS computes the inner product between the query vector and all 131 stored vectors in milliseconds. The top-k highest scoring chunks are returned.
3	**Apply Threshold**	Chunks below the similarity threshold are discarded. Concept queries use 0.20 (lower — because natural language scores lower against technical docs). Code queries use 0.30.
4	**Format Context**	The surviving chunks are formatted into a readable context string and inserted into the LLM prompt.

Why two different thresholds? When a user asks 'what is OEOmega?' (concept), the phrasing is general and the FAISS score will be lower even for highly relevant chunks. Concept queries use 0.20 so they don't get rejected unfairly. Code queries like 'generate conformers for aspirin' are more technical and score higher — they use 0.30 for stricter relevance.

05

LLM Intent Classifier — Smart Routing

After the keyword filter passes a request, a fast GPT-4o-mini call classifies the intent into exactly one of four categories. The model is given a system prompt that defines each category precisely, and it responds with a single word. The entire call uses about 10 output tokens — it is intentionally minimal.

Intent	Example Query	What Happens
GREETING	'hello', 'thanks'	Hardcoded instant response (already handled before this step in most cases)
CONVERSATION	'what is OEOmega?', 'explain sampling modes'	Path A: natural language response, no forced code block
CODE	'write code to generate conformers', 'show me an example'	Path B: code generation with Layer 3 validation and retry
OFF_TOPIC	'capital of France', 'who is Elon Musk'	Fallback response, no LLM generation

The Fallback Classifier

If the OpenAI API call fails for any reason (network issue, timeout, rate limit), the system falls back to a keyword-based classifier. This ensures the chatbot never goes completely dark — it degrades gracefully rather than crashing.

Context-Aware Classification

The classifier receives the last two messages from conversation history alongside the current query. This means 'verify this code' is correctly classified as CONVERSATION because the classifier can see that 'this code' refers to something from the previous turn. Without history, it might wrongly route it toward code generation.

06

Path A — Answering Concept Questions

When the intent is CONVERSATION, the system takes the gentler path. The goal is a natural, helpful explanation — not a forced code dump. The system prompt is written like instructions to a knowledgeable colleague, not a code generator.

What the Prompt Tells GPT

1	**Respond naturally**	Answer like a helpful colleague. No robotic structure.
2	**No code unless asked**	Do not include a code block unless the user explicitly requests one.
3	**Match response length**	Short factual questions get 3-5 sentences. Conceptual questions get 2-3 short paragraphs.
4	**Name real APIs**	Always mention actual class names (OEOmegaOptions, OEFlipperOptions) with concrete parameter examples.
5	**Yes/No questions**	Start with Yes or No, then explain in 2-4 sentences.
6	**Verify code**	If the user asks to check code, analyse it step by step and give a direct verdict.

Temperature = 0.4 — slightly higher than the code path (0.2) to allow for more natural, varied phrasing. Max tokens: 800. The conversational prompt is injected as a system message so it persists across the full message history.

07

Path B — Code Generation with Layer 3 Validation

Code generation is where things get interesting. GPT is good at writing Python, but it is also very capable of inventing function names that do not exist. In a domain-specific toolkit like OpenEye Omega, a hallucinated API call is worse than no code at all — a researcher could waste hours debugging something that simply does not exist. Layer 3 exists to catch exactly this.

The 5-Step OpenEye Pattern

Every valid Omega Toolkit script follows the same structure. The Layer 3 validator checks that all five components are present before accepting the output.

Step	What It Does	Example
1. Imports	Load the OpenEye modules	`from openeye import oechem, oeomega`
2. Streams	Open input/output molecule files	`ifs = oechem.oemolistream('in.sdf')`
3. Options	Configure the Omega settings	`opts = oeomega.OEOmegaOptions()`
4. Build	Generate the conformers	`ret_code = omega.Build(mol)`
5. Error Check	Handle failure return codes	`OEOmegaReturnCode_Success`

The Three Validation Checks

Check	How It Works	Catches
Syntax Check	ast.parse() — Python's own parser	Unclosed parentheses, missing colons, indentation errors
Pattern Check	Regex against 5 patterns — passes if 3 or more match	Code that has no OpenEye imports, no Build() call, no error handling
API Name Check	Every oeomega.X and oechem.X is checked against a curated allowlist	Hallucinated names like oeomega.OEFakeConformerBuilder()

The Retry Loop

If a check fails, the failure reason is converted into a clear, actionable instruction and appended to the next generation attempt. For example: 'Your previous code used unknown API names that may be hallucinated: OEFakeHallucinatedCall. Use only documented OpenEye Omega Toolkit APIs.' GPT receives this alongside the original question and generates again. The loop runs up to 5 times. In practice, almost all queries pass on the first or second attempt.

> **Temperature = 0.2** for code generation — low enough to keep responses deterministic and correct, but not zero (which can cause repetitive failures on retries). Max tokens: 2048 to allow for complete code blocks.

08

Conversation Memory — History and Summarization

A chatbot that forgets what was said two messages ago is frustrating. This system handles memory in two ways: a sliding window for recent turns, and automatic summarization for older turns that would otherwise be lost.

Sliding Window (Last 6 Turns)

Every API call includes the last 6 messages from the conversation. The frontend sends the full history in each request, and the backend slices the most recent 6 turns before constructing the LLM message array. This keeps the context window manageable and costs predictable.

Automatic Summarization (Older Turns)

When the conversation grows beyond 6 turns, the older messages don't simply vanish. They are summarized into 2-3 sentences by a separate GPT call. The summary captures what the user was working on, what code was generated, and any decisions made. This summary is prepended to the LLM context as a system message, so the assistant always knows the broader story of the conversation.

Summary Caching

Generating a summary costs tokens. The system caches summaries in memory using an MD5 hash of the message content as the key. If the same slice of history appears again (because no new old messages were added), the cached summary is returned instantly. The cache holds up to 200 entries before evicting the oldest.

Follow-Up Query Enrichment

Short follow-up queries like 'now use dense sampling instead' score poorly against the FAISS index because they contain almost no domain-specific terms. The system detects follow-up signals ('now', 'also', 'change it', 'same', 'this') and enriches the FAISS query by appending all prior user messages as context. This improved retrieval scores from 0.26 to 0.61 in testing.

Persistent History via Supabase

Beyond the in-memory sliding window, every chat turn is written to the chat_history table in Supabase. When a user reloads the page, their session_id (stored in localStorage) is used to fetch previous turns from GET /api/history/:session_id. The frontend reconstructs the message objects — including code blocks — and displays them with a 'History restored' toast notification.

09

Knowledge Base — User-Uploaded Context

The built-in FAISS index covers the official OpenEye documentation. But researchers often have their own notes, internal protocols, or custom code patterns. The Knowledge Base feature lets users upload their own context that the assistant will use alongside the official docs.

Adding Knowledge

Users can paste text directly (with a source name for reference) or upload files. Supported file types include PDF (text extracted using pypdf), plain text files, Markdown files, and images. Images are processed through GPT-4o vision to extract any visible text or describe diagrams technically.

1	**Text Input**	User pastes text and gives it a source name in the Knowledge Base panel.
2	**Chunking**	The text is split into overlapping chunks of 400 characters using a recursive splitter that respects paragraph and sentence boundaries.
3	**Embedding**	Each chunk is embedded using text-embedding-3-small (same model as the FAISS index) to produce a 1536-dim vector.
4	**Storage**	Chunks are stored in Supabase's knowledge_chunks table with the session_id, source name, text, and embedding vector (as JSONB, not pgvector).
5	**Retrieval**	At query time, _retrieve_knowledge() fetches all chunks for the session, embeds the query, computes cosine similarity in Python using numpy, and returns the top 3 chunks above a 0.25 threshold.
6	**Merge**	Retrieved knowledge chunks are prepended to the prompt context under a [User Knowledge Base] section, above the [Retrieved Documentation] from FAISS.

Why JSONB instead of pgvector? Supabase supports pgvector for vector similarity search in SQL, but that requires a specific database extension and schema setup. Since the similarity computation happens in Python anyway (using numpy), storing the embedding as a plain JSONB array is simpler and requires no additional database configuration. For small numbers of chunks per session, the performance difference is negligible.

10

Supabase — Persistence and Analytics

Supabase is the persistence layer. It runs as a managed PostgreSQL database in the cloud. The system uses four tables. All database writes are fire-and-forget — they don't block the response to the user, so database slowness never affects chat latency.

Table	Key Columns	Purpose
chat_history	session_id, role, content, created_at	Stores every user and assistant message so chat can be restored on page reload.
queries_log	session_id, message, intent, is_fallback, has_code, attempts, faiss_score, latency_ms	Analytics data for every query. Powers the Analytics dashboard.
feedback	session_id, message_id, feedback ('up'/'down')	Thumbs up/down ratings from users. Shown in Analytics as helpful rate.
knowledge_chunks	session_id, source, text, embedding (JSONB)	User-uploaded knowledge chunks with their embedding vectors for cosine similarity search.

Graceful Degradation

The system is designed to work even without Supabase configured. If SUPABASE_URL or SUPABASE_KEY are missing from the environment, every database function returns silently: chat history returns an empty list, analytics returns zero counts, feedback writes return {ok: false}. The core chat functionality always works. This makes local development easy — only the OpenAI key is required.

11

Frontend Architecture — The React Interface

The frontend is a single-page React application built with Vite. It uses Tailwind CSS (loaded via CDN — no build step needed for styles), framer-motion for animations, lucide-react for icons, and highlight.js for code syntax highlighting. The final bundle is 324 kB.

Component Structure

Component	What It Renders
App.jsx	Global state (session, messages, view), session init on mount, history restoration, toast notifications.
LeftPanel.jsx	Sidebar with session list, New Chat button, and nav items for Chat / Analytics / Knowledge Base.
MiddlePanel.jsx	Routes between ChatScreen, AnalyticsDashboard, and KnowledgePanel based on active view.
ChatScreen.jsx	Header with Export button, MessageList, and sticky InputBar at the bottom.
MessageList.jsx	Renders user messages, bot messages (with attempts badge), fallback boxes, and the ThinkingIndicator.
CodeBlock.jsx	Syntax-highlighted code using highlight.js + a Copy button.
InputBar.jsx	Auto-growing textarea, Paperclip and Mic icons (with 'Coming soon' tooltips), Send button.
AnalyticsDashboard.jsx	Total Queries, Code Requests, Guardrail Blocks, Avg Latency, Response Quality (helpful rate), recent queries table.
KnowledgePanel.jsx	Text paste area, drag-and-drop file upload, list of uploaded sources with delete buttons.
WelcomeScreen.jsx	Shown before first message: branding, tagline, 2x2 suggestion cards.
TextShimmer.jsx	Animated gradient sweep on the ThinkingIndicator text using framer-motion.

Session Persistence

When the app loads, it checks localStorage for a key called omega_session_id. If found, it reuses that session and fetches the chat history from Supabase. If not found, it generates a new UUID session_id, saves it to localStorage, and creates a placeholder session entry. When the user clicks New Chat, a brand new UUID is generated and saved — the old session history stays in the database.

12

Testing Strategy — 59 Tests, All Passing

The project has a full automated test suite that runs with pytest. All 59 tests pass consistently. Tests are split into three files, each targeting a different part of the system.

File	Tests	What's Covered
test_guardrails.py	29 tests	Layer 1 intent check (VALID/INVALID for 12 query types), Layer 2 retrieval confidence (custom thresholds), Layer 3 syntax/pattern/API name checks.
test_chat.py	28 tests	parse_llm_response, build_retrieval_query, _handle_conversational shortcuts, HTTP endpoint tests (with and without OPENAI_API_KEY), live pipeline tests for off-topic/concept/code/attempts.
test_health.py	2 tests	Health endpoint returns 200 with {status: ok}.

Live tests vs. mocked tests: Tests that require an OpenAI API key are automatically skipped if the key is not present. This lets the test suite run in CI environments without secrets. Tests that don't need the key (guardrail checks, response parsing, hardcoded conversational handlers) always run.

13

Improvements and What Could Be Built Next

The system works well, but there are clear directions where it could grow. Some are small quality-of-life improvements. Others would be significant architectural additions.

Near-Term Improvements

1	**Voice Input**	The InputBar has a Mic button that shows 'Coming soon'. The backend already has POST /api/transcribe which accepts audio via Whisper. Wiring the frontend to record and send audio would complete this feature with minimal backend work.
2	**File Attachments in Chat**	The Paperclip icon exists in the InputBar. The backend handles file uploads in the Knowledge Base panel. Allowing files to be attached directly in the chat input (processed inline as context for that single message) would make the workflow faster.
3	**pgvector for Knowledge Base**	Currently, cosine similarity for knowledge retrieval is computed in Python by loading all session chunks into memory. Using Supabase's pgvector extension would push this computation to the database — faster, scalable to thousands of chunks.
4	**Streaming Responses**	Currently, GPT generates the full response before anything appears in the chat. Using the OpenAI streaming API and Server-Sent Events would show tokens as they arrive, making the interface feel dramatically faster.
5	**Code Execution Sandbox**	The generated code currently shows in a read-only block. Adding a sandboxed Python executor (using a service like Pyodide in the browser, or a Docker container on the server) would let users run the generated code directly and see its output.
6	**Better Analytics Joins**	The feedback table uses session_id + message_id, but queries_log doesn't store message_id. This makes per-message feedback breakdown impossible without a schema change. Adding message_id to queries_log would unlock per-response quality metrics.

Larger Architectural Ideas

1	**Multi-Toolkit Support**	The RAG index currently contains only Omega Toolkit docs. The same architecture would work for OEDocking, OESpruce, or any other OpenEye toolkit by building separate FAISS indices and routing queries to the right one.
2	**Fine-Tuned Model**	Instead of relying on few-shot prompting and retrieval, a fine-tuned version of GPT-4o-mini trained on curated Omega Toolkit Q&A; pairs would reduce hallucinations further and potentially eliminate the need for Layer 3 retries.
3	**Active Learning from Feedback**	Thumbs up/down feedback is collected but currently only displayed. A pipeline that automatically flags low-rated responses for human review and uses them to improve the prompt or fine-tune the model would close the feedback loop.
4	**Multi-User Authentication**	Currently, any user can access any session by guessing a session_id (UUIDs make this hard but not impossible). Proper user authentication (OAuth, Supabase Auth) would isolate sessions and knowledge bases per authenticated user.

Quick Reference

All API Endpoints

Method	Endpoint	What It Does
GET	`/api/health`	Server health check
POST	`/api/chat`	Main chat endpoint — full pipeline
GET	`/api/history/:session_id`	Fetch past messages for a session
GET	`/api/analytics`	Query stats for last 24 hours
POST	`/api/feedback`	Record thumbs up or down for a response
POST	`/api/knowledge`	Add pasted text to the knowledge base
POST	`/api/knowledge-file`	Upload a PDF, text, or image file
GET	`/api/knowledge/:session`	List knowledge sources for a session
DELETE	`/api/knowledge/:chunk_id`	Delete a knowledge source (all its chunks)
POST	`/api/transcribe`	Transcribe audio using Whisper

Key Numbers

131	Documentation chunks in the FAISS index
1,536	Embedding dimensions (text-embedding-3-small)
5	Maximum Layer 3 retry attempts per code query
6	Message window injected into every LLM call
0.20 / 0.30	FAISS similarity thresholds (concept / code)
0.25	Knowledge base cosine similarity threshold

59	Automated tests (all passing)
200	In-memory summary cache capacity
324 kB	Frontend bundle size (gzipped)

How to run locally: /opt/anaconda3/envs/omega-tk-web/bin/python3 server/app.py then open http://localhost:8000 — only OPENAI_API_KEY is required.